

■ Enterprise Blueprint: Global Health Management System (GHMS)

****Author:**** Principal Software Architect, Google

****Version:**** 1.0.0

****Status:**** Draft / For Stakeholder Review

■ Project Overview

The ****Global Health Management System (GHMS)**** is a cloud-native, HIPAA-compliant, and highly scalable ecosystem designed to digitize and optimize the end-to-end healthcare experience. Unlike legacy monolithic systems, GHMS leverages a microservices architecture to decouple patient care, administrative operations, and financial workflows.

The platform serves as a unified "Source of Truth" for medical data, integrating real-time scheduling, secure electronic health records (EHR), automated pharmacy management, and an AI-ready analytics engine. It is designed for high availability (99.99%) and global scale, ensuring that whether it's a local clinic or a multi-national hospital chain, the performance remains deterministic.

■ Business Objective

1. ****Operational Efficiency:**** Reduce administrative overhead by 40% through automated scheduling and billing.
2. ****Patient Centricity:**** Provide a seamless digital experience for patients to manage their health journey.
3. ****Data Integrity & Security:**** Ensure 100% compliance with global healthcare standards (HIPAA, GDPR, HL7/FHIR).
4. ****Clinical Accuracy:**** Minimize human error in prescriptions and lab reports via real-time validation and history access.
5. ****Scalability:**** Support a horizontal growth model from 1,000 to 1,000,000+ concurrent users without architectural refactoring.

■ User Roles

* ****Patient:**** Register, book/manage appointments, view records, pay bills, receive notifications.

* ****Doctor:**** Manage clinical schedules, update EHRs, issue digital prescriptions, view patient history.

* ****Administrator:**** System configuration, staff management, department oversight, high-level reporting.

* ****Lab Technician:**** Upload diagnostic reports, manage test inventory, notify doctors/patients of results.

* ****Pharmacist:**** Process digital prescriptions, manage medication inventory, track dispensations.

* ****Billing/Finance Officer:**** Manage insurance claims, process invoices, generate financial audits.

■ Functional Requirements

Core Modules

- * **Identity & Access Management (IAM):** Secure OAuth2/OpenID Connect login with Role-Based Access Control (RBAC).
- * **Patient Portal:** Self-registration, profile management, and a dashboard for medical history.
- * **Appointment Engine:** Real-time scheduling with conflict resolution and "waitlist" logic.
- * **EHR (Electronic Health Records):** Secure storage for clinical notes, ICD-10 coded diagnoses, and treatment plans.
- * **Digital Pharmacy:** Automated stock alerts, prescription fulfillment tracking, and expiration monitoring.
- * **Laboratory Information System (LIS):** Test ordering, result entry, and automated PDF report generation.
- * **Billing & Payments:** Multi-gateway support (Stripe/PayPal), insurance verification, and automated invoicing.
- * **Notification Engine:** Multi-channel (Email, SMS, Push) alerts for reminders and emergencies.

■ Non-Functional Requirements

1. **Security:** AES-256 encryption for data at rest; TLS 1.3 for data in transit. Zero-trust architecture.
2. **Compliance:** Strict adherence to HIPAA (USA) and GDPR (EU) for data privacy.
3. **Availability:** 99.99% (Multi-AZ deployment).
4. **Performance:** API latency $\leq 200\text{ms}$ for 95th percentile (P95).
5. **Scalability:** Auto-scaling via Kubernetes based on CPU/Memory utilization.
6. **Auditability:** Every record modification must be logged with a timestamp and actor ID (Immutable Audit Logs).

■ Suggested Technology Stack

| Component | Technology |

| :--- | :--- |

| **Frontend (Web)** | React.js, Next.js (SSR), TailwindCSS, Redux Toolkit |

| **Frontend (Mobile)** | Flutter (Cross-platform iOS/Android) |

| **Backend (API)** | Node.js (TypeScript) with NestJS Framework |

| **Database (Relational)** | PostgreSQL (Cloud SQL) for ACID compliance |

| **Database (Caching)** | Redis for session management and schedule caching |

| **Storage (Object)** | Google Cloud Storage (for Lab Reports/Scans) |

| **Infrastructure** | Google Cloud Platform (GCP), GKE (Kubernetes) |

| **Auth/IAM** | Firebase Auth or Okta (OIDC/SAML) |

| **CI/CD** | GitHub Actions & Google Cloud Build |

| **Monitoring** | Prometheus, Grafana, and Google Cloud Logging |

■ Suggested Folder Structure

Using a ****Domain-Driven Design (DDD)**** modular approach.

```text

/ghms-monorepo

■ ■ ■ /apps

■ ■ ■ ■ /web-patient # Next.js Patient Portal

■ ■ ■ ■ /web-admin # Next.js Admin/Staff Dashboard

■ ■ ■ ■ /mobile-app # Flutter Mobile App

■ ■ ■ /services

■ ■ ■ ■ /auth-service # Authentication & RBAC

■ ■ ■ ■ /patient-service # Patient Profiles & Records

■ ■ ■ ■ /booking-service # Appointment Logic

■ ■ ■ ■ /finance-service # Billing & Payments

■ ■ ■ ■ /notif-service # SMS/Email/Push Workers

■ ■ ■ /libs

■ ■ ■ ■ /common-dto # Shared TypeScript Interfaces

■ ■ ■ ■ /database # TypeORM/Prisma Schemas

■ ■ ■ /k8s # Kubernetes Manifests

■ ■ ■ README.md

```

■ REST API Endpoints (Samples)

| Method | Endpoint | Purpose |

| :--- | :--- | :--- |

| `POST` | `/api/v1/auth/register` | Register new patient/staff |

| `POST` | `/api/v1/auth/login` | Secure login & JWT issuance |

| `GET` | `/api/v1/appointments` | Fetch available slots or user bookings |

| `POST` | `/api/v1/appointments/book` | Create a new appointment booking |

| `GET` | `/api/v1/patients/:id/records` | Retrieve clinical history (Doctor/Patient only) |

| `POST` | `/api/v1/prescriptions` | Issue new digital prescription |

| `POST` | `/api/v1/payments/checkout` | Initiate payment for services |

| `GET` | `/api/v1/inventory/stocks` | Check pharmacy/lab supply levels |

■ Database Design (Relational Schema)

Table: Users

- * `id` (UUID, PK)
- * `email` (String, Unique)
- * `password\_hash` (String)
- * `role` (Enum: PATIENT, DOCTOR, ADMIN, etc.)
- * `created\_at` (Timestamp)

Table: Patients

- * `id` (UUID, PK)
- * `user\_id` (FK -> Users)
- * `dob` (Date)
- * `blood\_group` (String)
- * `emergency\_contact` (String)

Table: Appointments

- * `id` (UUID, PK)
- * `patient\_id` (FK -> Patients)
- * `doctor\_id` (FK -> Staff)
- * `status` (Enum: PENDING, CONFIRMED, COMPLETED, CANCELLED)
- * `scheduled\_at` (Timestamp)

Table: Medical_Records

- * `id` (UUID, PK)
- * `patient\_id` (FK -> Patients)
- * `doctor\_id` (FK -> Staff)
- * `diagnosis` (Text)
- * `prescription\_json` (JSONB)
- * `attachments\_url` (Array)

■ Deployment Architecture

1. **Traffic Entry:** Google Cloud Load Balancer (GCLB) with Cloud Armor (WAF) for DDoS protection.
2. **Orchestration:** Google Kubernetes Engine (GKE) running Dockerized Microservices.
3. **Service Mesh:** Istio for mTLS (internal service encryption) and traffic splitting.
4. **Database:** Cloud SQL (PostgreSQL) with High Availability (Failover Replica).
5. **Secrets:** Google Secret Manager for API keys and DB credentials.
6. **CDN:** Cloud CDN for serving static frontend assets and medical images.

■ Development Roadmap

Week 1: Foundations & IAM

- * Setup GCP Project & GKE Cluster.
- * Implement Auth Service (OAuth2/RBAC).
- * Initialize Database Schema.
- * Setup CI/CD Pipelines.

Week 2: Patient & Staff Core

- * Patient Profile Management.
- * Staff/Doctor Onboarding.
- * Department and Inventory Setup.
- * Basic Web UI Shell.

Week 3: Appointment Engine

- * Calendar Logic & Conflict Validation.
- * Doctor Availability Management.
- * Booking Workflow (Web/Mobile).
- * Initial SMS/Email Notification triggers.

Week 4: Clinical Modules (EHR & Lab)

- * Medical Record Entry UI.
- * Lab Result Upload & Processing.
- * Digital Prescription Generation (PDF).
- * Search/Filter for Clinical History.

Week 5: Finance & Analytics

- * Payment Gateway Integration (Stripe).
- * Invoicing and Billing Logic.
- * Admin Dashboard with Analytics (BigQuery integration).
- * Audit Logging System.

Week 6: Quality Assurance & Launch

- * End-to-End Testing (Cypress).
- * Load Testing (Locust) and Security Pen-testing.
- * HIPAA Compliance Audit.
- * Production Deployment (Blue/Green Deployment).

Approval:

Date: October 26, 2023

Signed: [Principal Architect Name]